

Министерство образования и науки Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2
по курсу «Логика и Основы Алгоритмизации
в Инженерных Задачах»
на тему «Оценка времени выполнения программ»

Выполнил:

студент группы 25BVB3

Радушев И.А.

Приняли:

к.п.н., доцент Деев М.В.

к.т.н., доцент Юрова Т.В.

Пенза 2026

Цель работы: изучить способы оценки времени выполнения программ на языке C, аналитически определить временную сложность алгоритмов и практически подтвердить полученные оценки с помощью экспериментальных замеров времени.

Задачи:

1. Изучить функции библиотеки `time.h`, применяемые для измерения времени выполнения программ.
2. Реализовать программу для перемножения квадратных матриц различных размеров.
3. Аналитически определить временную сложность алгоритма умножения матриц.
4. Провести экспериментальные замеры времени выполнения умножения матриц.
5. Подобрать коэффициент для построения теоретической зависимости времени от размера входных данных.
6. Сравнить теоретические и экспериментальные результаты в таблице и на графике.
7. Реализовать и исследовать алгоритмы сортировки Shell sort, Quick sort и стандартную функцию `qsort`.
8. Провести замеры времени сортировки на разных типах входных данных: случайных, возрастающих, убывающих и возрастающе-убывающих.
9. Сравнить результаты работы алгоритмов сортировки на различных объемах данных.

Ход работы:

На первом этапе была реализована программа для оценки времени выполнения алгоритма умножения квадратных матриц. Для хранения матриц использовались динамические одномерные массивы, представляющие матрицы размера $n \times n$. Матрицы заполнялись случайными числами, после чего выполнялось их перемножение. Для измерения времени работы использовалась функция `clock()` из библиотеки `time.h`. Замеры проводились для матриц различных размеров. Для уменьшения погрешности на малых размерах вычисление могло повторяться несколько раз, после чего рассчитывалось среднее

время одного запуска. Теоретическая сложность алгоритма была определена как $O(n^3)$, так как в программе используются три вложенных цикла, каждый из которых зависит от размера матрицы n . Для сравнения теоретических и экспериментальных данных был подобран коэффициент k , после чего теоретическое время вычислялось по формуле $T(n) = k * n^3$.

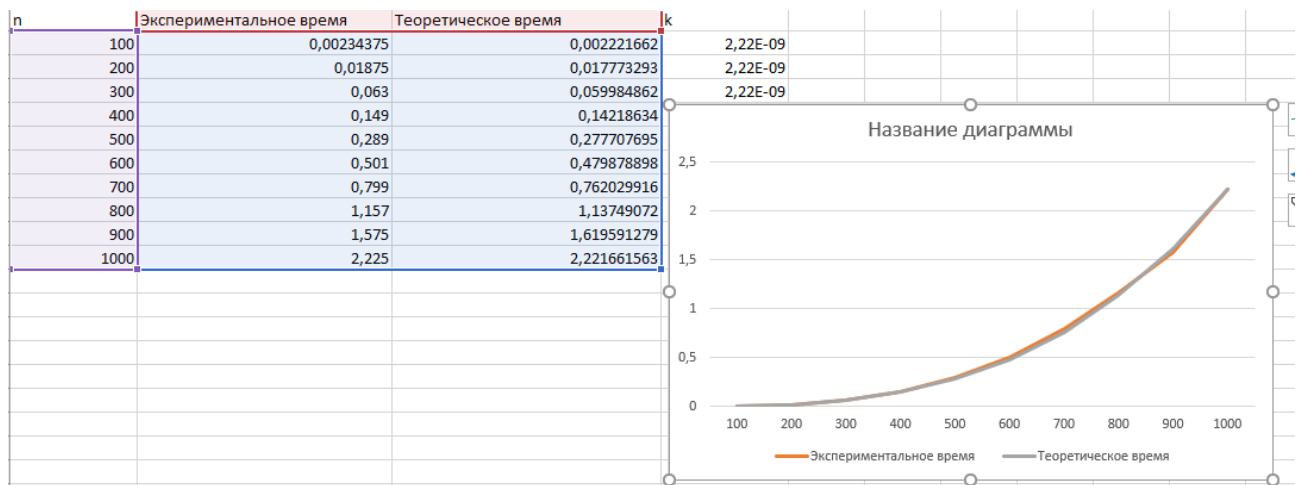
Task 1. Matrix multiplication
Analytical complexity: $O(n^3)$

Selected coefficient $k = 2.221661562597e-009$

n	$f(n) = n^3$	theoretical, sec	experiment, sec
100	1000000	0.002221662	0.002343750
200	8000000	0.017773293	0.018750000
300	27000000	0.059984862	0.063000000
400	64000000	0.142186340	0.149000000
500	125000000	0.277707695	0.289000000
600	216000000	0.479878898	0.501000000
700	343000000	0.762029916	0.799000000
800	512000000	1.137490720	1.157000000
900	729000000	1.619591279	1.575000000
1000	1000000000	2.221661563	2.225000000

Рисунок 1 - Таблица теоретического и экспериментального времени выполнения алгоритма умножения матриц

Оценим сложность алгоритма умножения двух квадратных матриц размера $n \times n$. В программе используются три вложенных цикла. Первый цикл перебирает строки первой матрицы и выполняется n раз. Второй цикл перебирает элементы строки первой матрицы и также выполняется n раз. Третий цикл перебирает столбцы второй матрицы и выполняется n раз. Внутри самого вложенного цикла выполняется операция умножения и сложения: $c[i][j] += a[i][k] * b[k][j]$. Так как каждый из трех вложенных циклов выполняется n раз, количество основных операций пропорционально $n \cdot n \cdot n = n^3$. Следовательно, временная сложность алгоритма умножения матриц равна $O(n^3)$.



Сравнение теоретического и экспериментального времени умножения матриц

На втором этапе работы были исследованы алгоритмы Shell sort, Quick sort и стандартная функция qsort. Измерения проводились для массивов размером 100, 1000 и 10000 элементов. Использовались случайные, монотонно возрастающие, монотонно убывающие и возрастающе-убывающие наборы данных.

Алгоритм Shell sort выполняет сортировку элементов с постепенно уменьшающимся шагом. В программе начальный шаг равен половине размера массива, после чего он уменьшается в два раза. При неудачном расположении элементов количество сравнений и перемещений пропорционально квадрату размера массива. Поэтому сложность используемой реализации в худшем случае составляет $O(n^2)$.

Алгоритм Quick sort разделяет массив относительно опорного элемента. Если массив делится на приблизительно равные части, на каждом уровне выполняется $O(n)$ операций, а количество уровней составляет $\log n$. Средняя временная сложность равна $O(n \log n)$. Если разбиение оказывается крайне неравномерным, сложность может увеличиться до $O(n^2)$.

Функция qsort является стандартной функцией библиотеки языка C. Конкретный алгоритм ее работы зависит от реализации используемой библиотеки. Для практического сравнения принимается ожидаемая сложность $O(n \log n)$.

Для повышения точности каждый замер выполнялся несколько раз, после чего вычислялось среднее время одной сортировки. Восстановление исходного массива производилось до начала измерения, поэтому время копирования не учитывалось в результате.

n	Data	Algorithm	Time, sec
100	Random	Shell sort	0.000000854
100	Random	Quick sort	0.000000549
100	Random	qsort	0.000002319
100	Ascending	Shell sort	0.000000610
100	Ascending	Quick sort	0.000000610
100	Ascending	qsort	0.000001099
100	Descending	Shell sort	0.000000793
100	Descending	Quick sort	0.000000397
100	Descending	qsort	0.000001251
100	Ascending-descending	Shell sort	0.000000732
100	Ascending-descending	Quick sort	0.000001007
100	Ascending-descending	qsort	0.000003235
1000	Random	Shell sort	0.00015625
1000	Random	Quick sort	0.000009766
1000	Random	qsort	0.000033203
1000	Ascending	Shell sort	0.000009888
1000	Ascending	Quick sort	0.000003967
1000	Ascending	qsort	0.000017578
1000	Descending	Shell sort	0.000013184
1000	Descending	Quick sort	0.000004761
1000	Descending	qsort	0.000019287
1000	Ascending-descending	Shell sort	0.000011719
1000	Ascending-descending	Quick sort	0.000074219
1000	Ascending-descending	qsort	0.000057617
10000	Random	Shell sort	0.000781250
10000	Random	Quick sort	0.000460938
10000	Random	qsort	0.000765625
10000	Ascending	Shell sort	0.000154297
10000	Ascending	Quick sort	0.000056641
10000	Ascending	qsort	0.000207031
10000	Descending	Shell sort	0.000175781
10000	Descending	Quick sort	0.000056641
10000	Descending	qsort	0.000230469
10000	Ascending-descending	Shell sort	0.000166016
10000	Ascending-descending	Quick sort	0.006062500
10000	Ascending-descending	qsort	0.000843750

Рисунок 3 — Результаты измерения времени выполнения алгоритмов сортировки

По результатам экспериментальных измерений были построены диаграммы, показывающие зависимость времени выполнения алгоритмов Shell sort, Quick sort и qsort от типа входных данных. Сравнение проводилось отдельно для массивов размером 100, 1000 и 10000 элементов. Время выполнения алгоритмов указано в секундах.

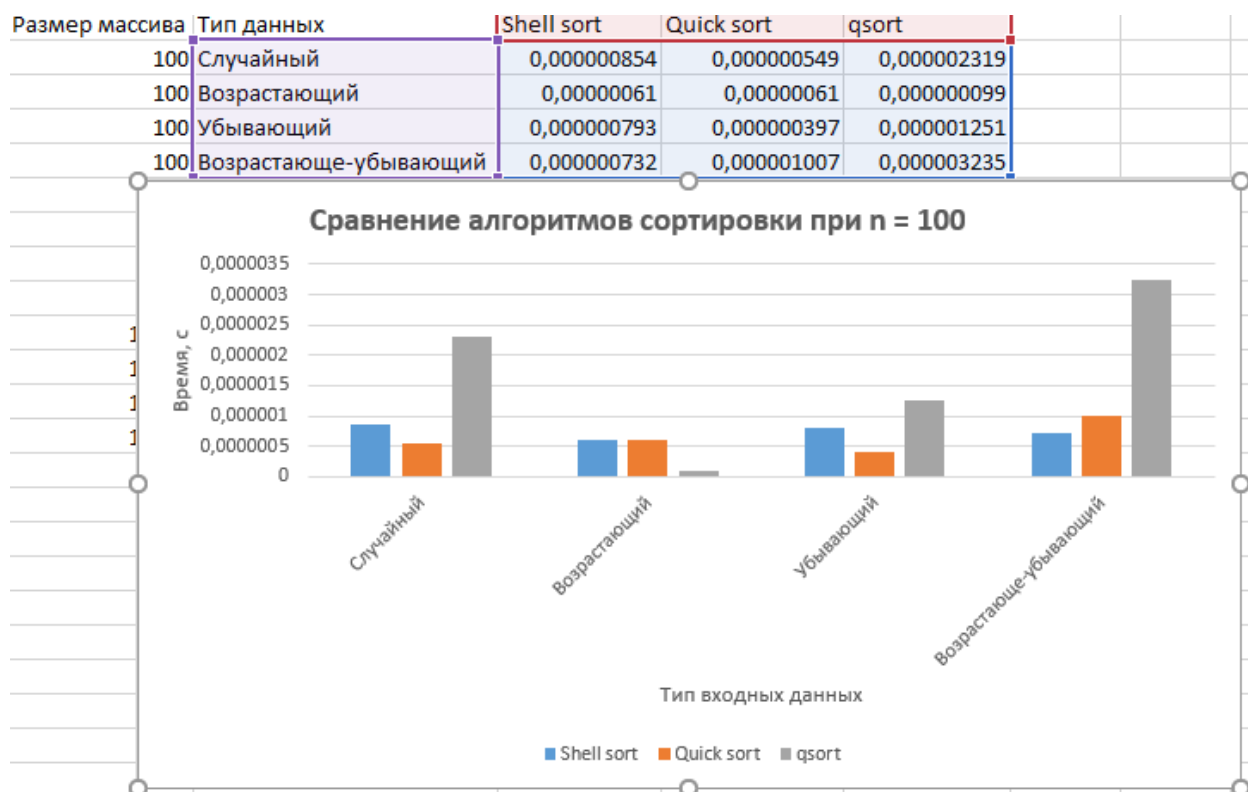


Рисунок 4 — Сравнение времени сортировки массивов размером 100 элементов

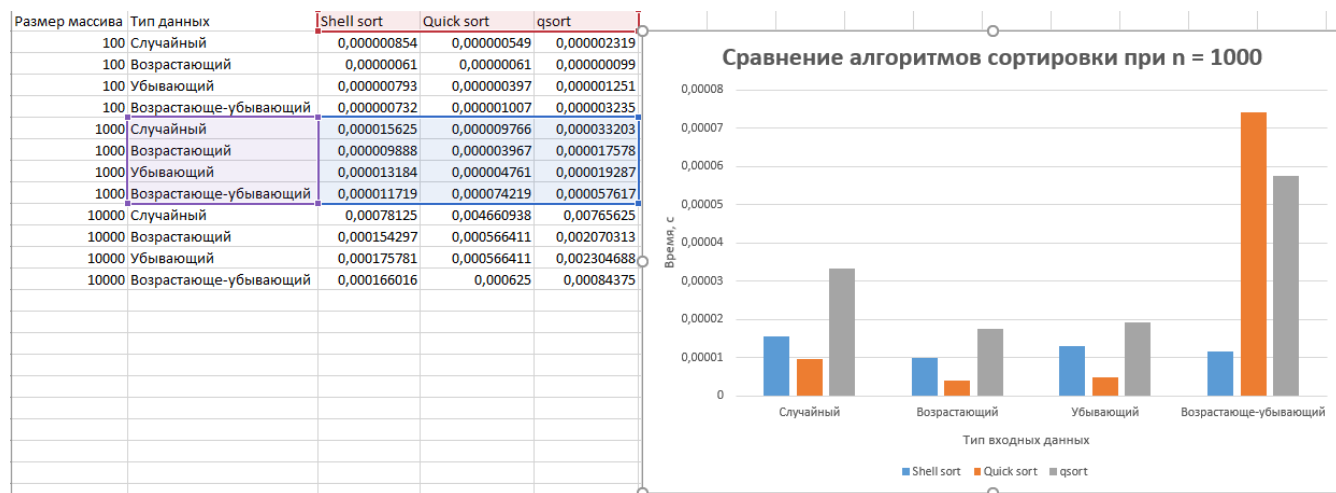


Рисунок 5 — Сравнение времени сортировки массивов размером 1000 элементов

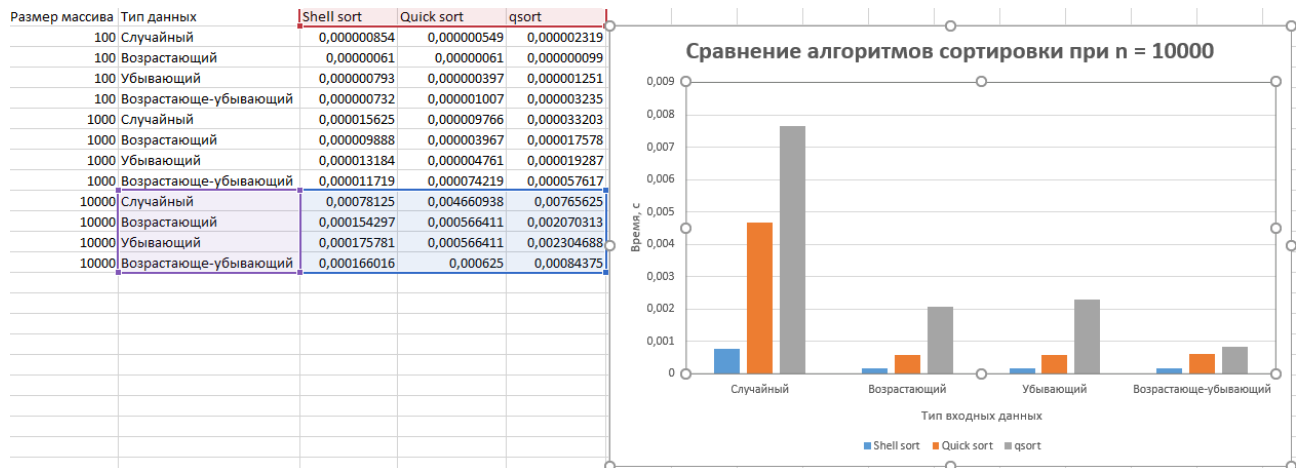


Рисунок 6 — Сравнение времени сортировки массивов размером 10000 элементов

Анализ экспериментальных результатов показал, что с увеличением размера массива время выполнения всех исследованных алгоритмов возрастает. Для массивов из 100 элементов различия между алгоритмами невелики, поскольку продолжительность одной сортировки очень мала и результат зависит от точности измерения времени.

Для массивов из 1000 элементов Quick sort показал наименьшее время на случайных, возрастающих и убывающих данных. На возрастающе-убывающем массиве быстрее работал Shell sort. Это показывает, что время сортировки зависит не только от количества элементов, но и от их начального расположения.

Для массивов из 10000 элементов наименьшее время во всех проведенных измерениях показал Shell sort. Особенно быстро он обработал частично упорядоченные массивы. Quick sort занял второе место, а стандартная функция qsort в большинстве экспериментов работала медленнее. Более продолжительная работа qsort может быть связана с особенностями библиотечной реализации и дополнительными вызовами функции сравнения.

Полученные значения времени могут немного изменяться при повторных запусках программы, так как на измерения влияют загрузка процессора, работа операционной системы и точность системного таймера. Однако общая тенденция роста времени при увеличении объема данных сохраняется.

Вывод: В ходе лабораторной работы были изучены методы аналитической и экспериментальной оценки времени выполнения программ.

Для алгоритма умножения квадратных матриц установлена временная сложность $O(n^3)$. Теоретическое время рассчитывалось по формуле $T(n) = k \cdot n^3$. Построенный график показал близость теоретических и экспериментальных результатов, что подтвердило правильность аналитической оценки.

Также были исследованы алгоритмы Shell sort, Quick sort и стандартная функция qsort. Измерения проводились на массивах размером 100, 1000 и 10000 элементов с различным расположением данных. Эксперименты показали, что время сортировки зависит как от размера массива, так и от исходного порядка элементов. В проведённых измерениях Quick sort показал хорошие результаты для массивов среднего размера, а Shell sort оказался наиболее быстрым для массивов из 10000 элементов.

Таким образом, аналитическая оценка позволяет определить характер роста времени выполнения алгоритма, а экспериментальные измерения позволяют сравнить конкретные реализации с учётом входных данных и условий выполнения программы.